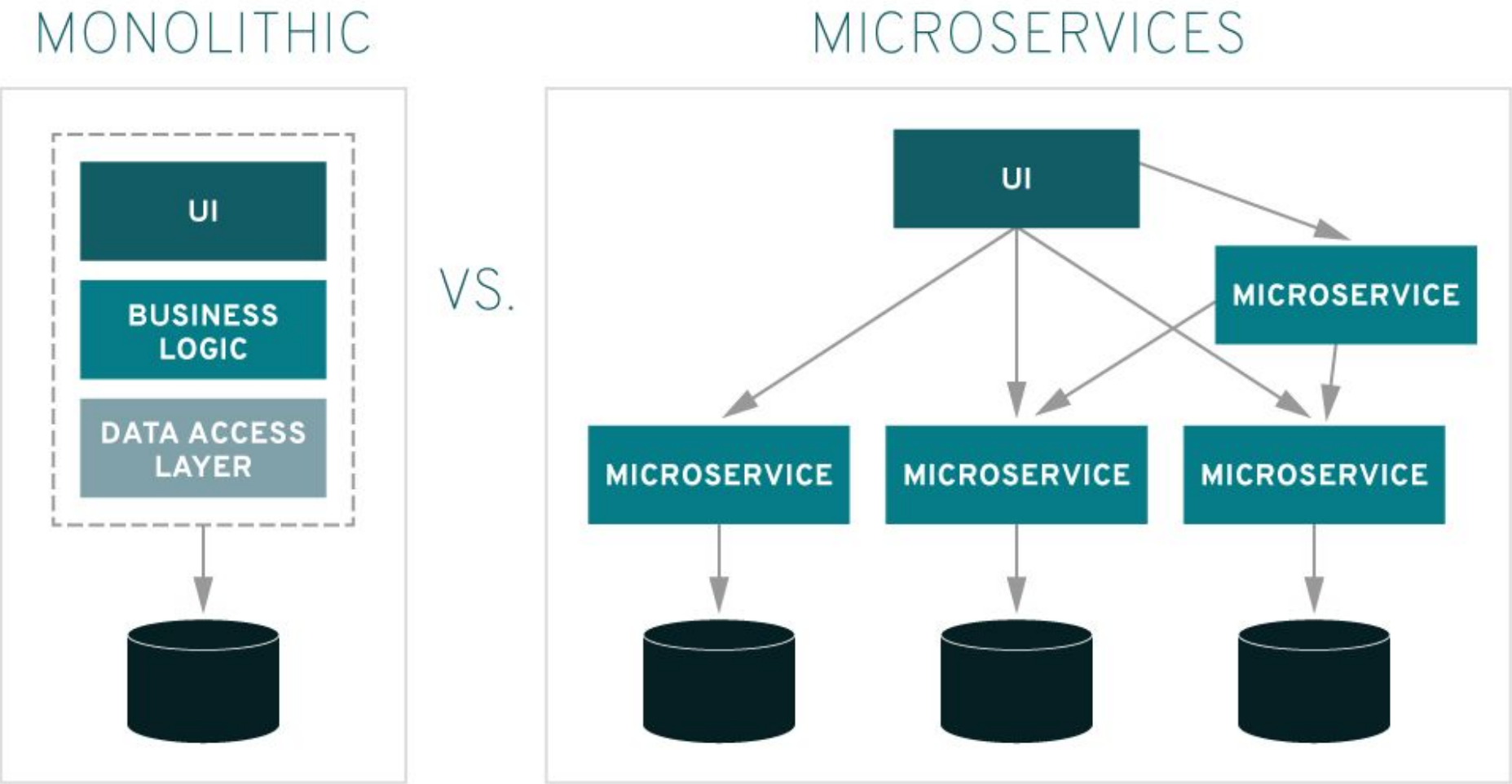
Hochschule Konstanz
Fakultät Informatik

Software Architecture

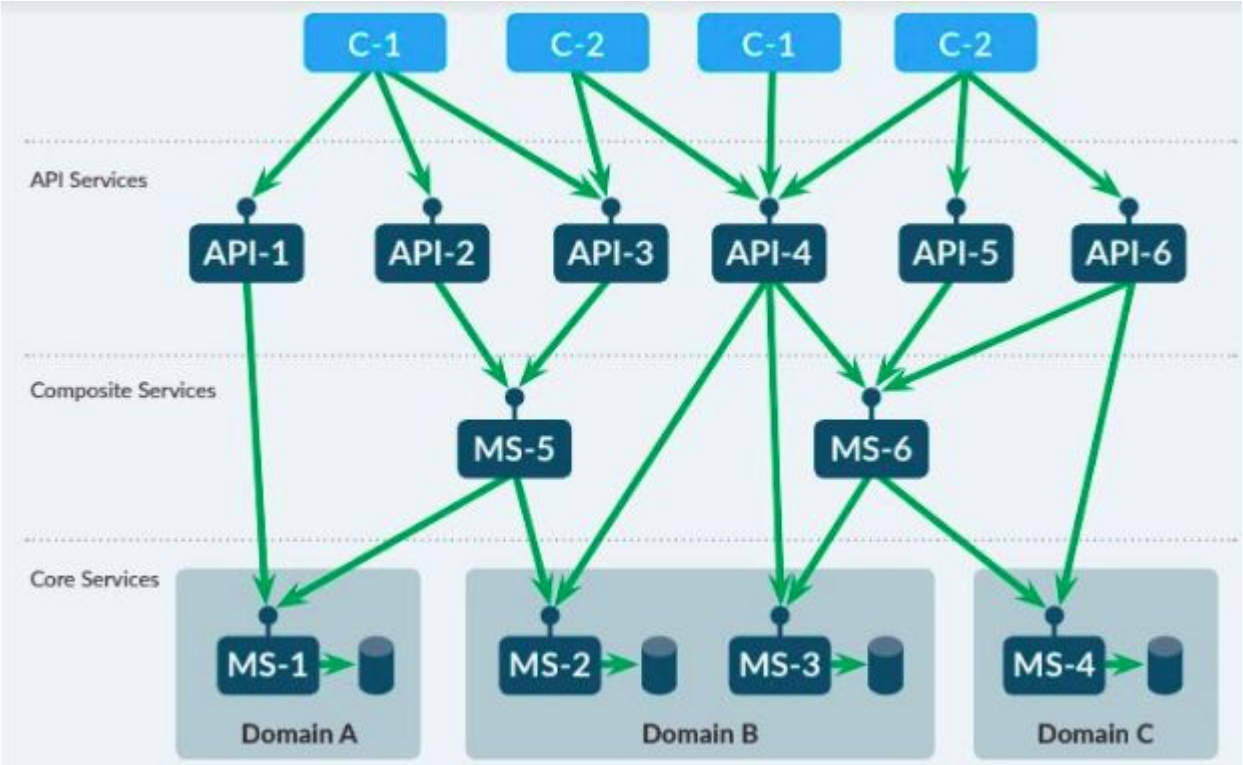
11 - Kafka

Prof. Dr. Marko Boger

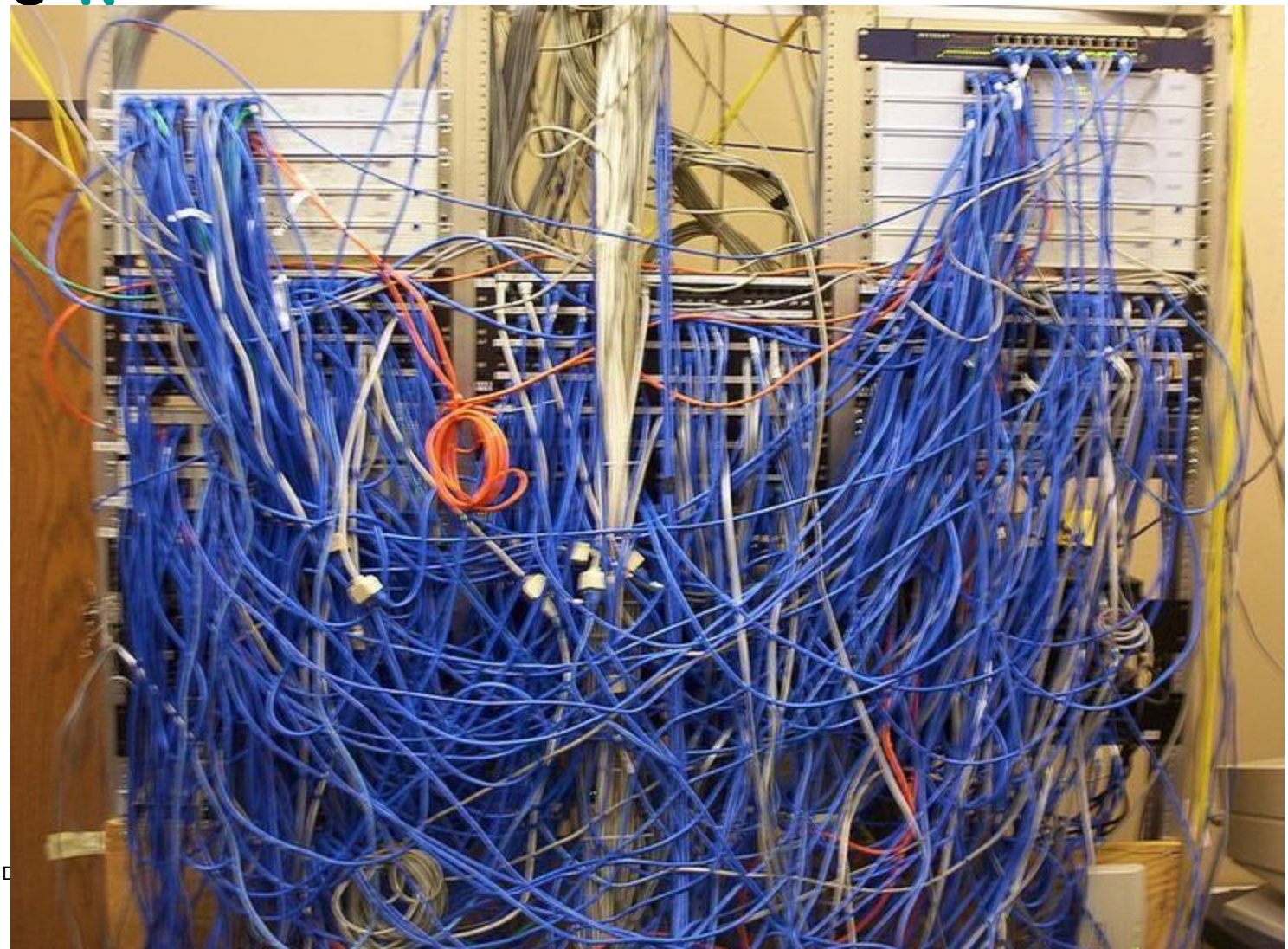
Monolith vs Microservices



Deep Microservices

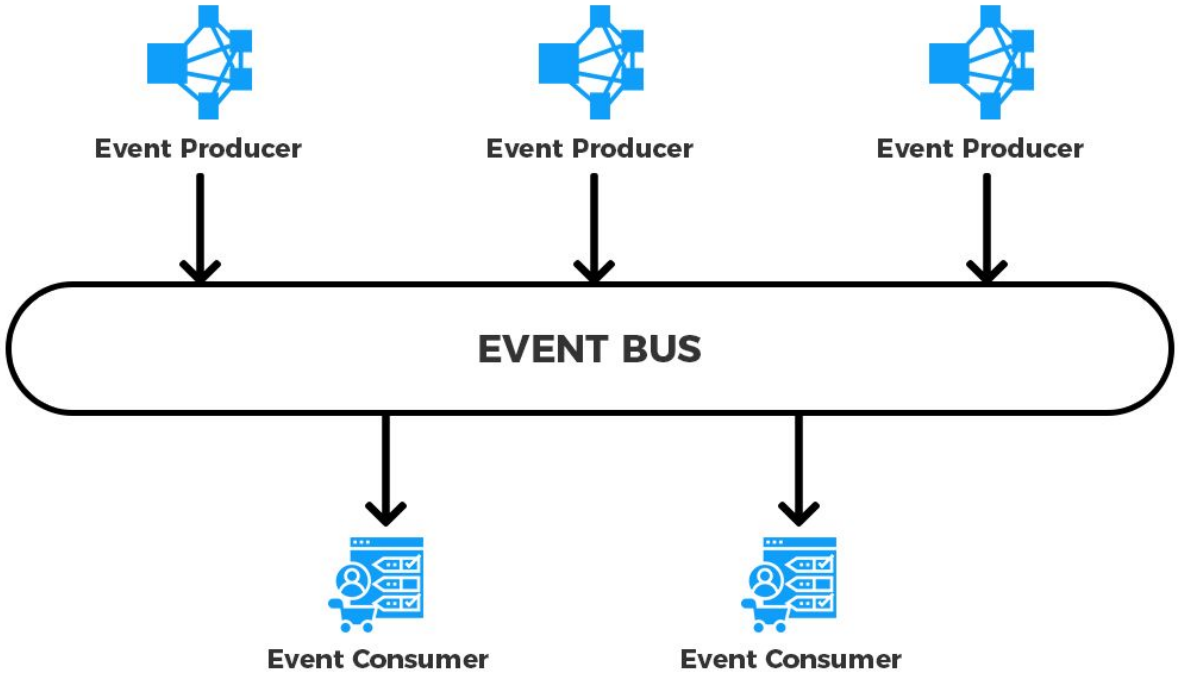


The Problem



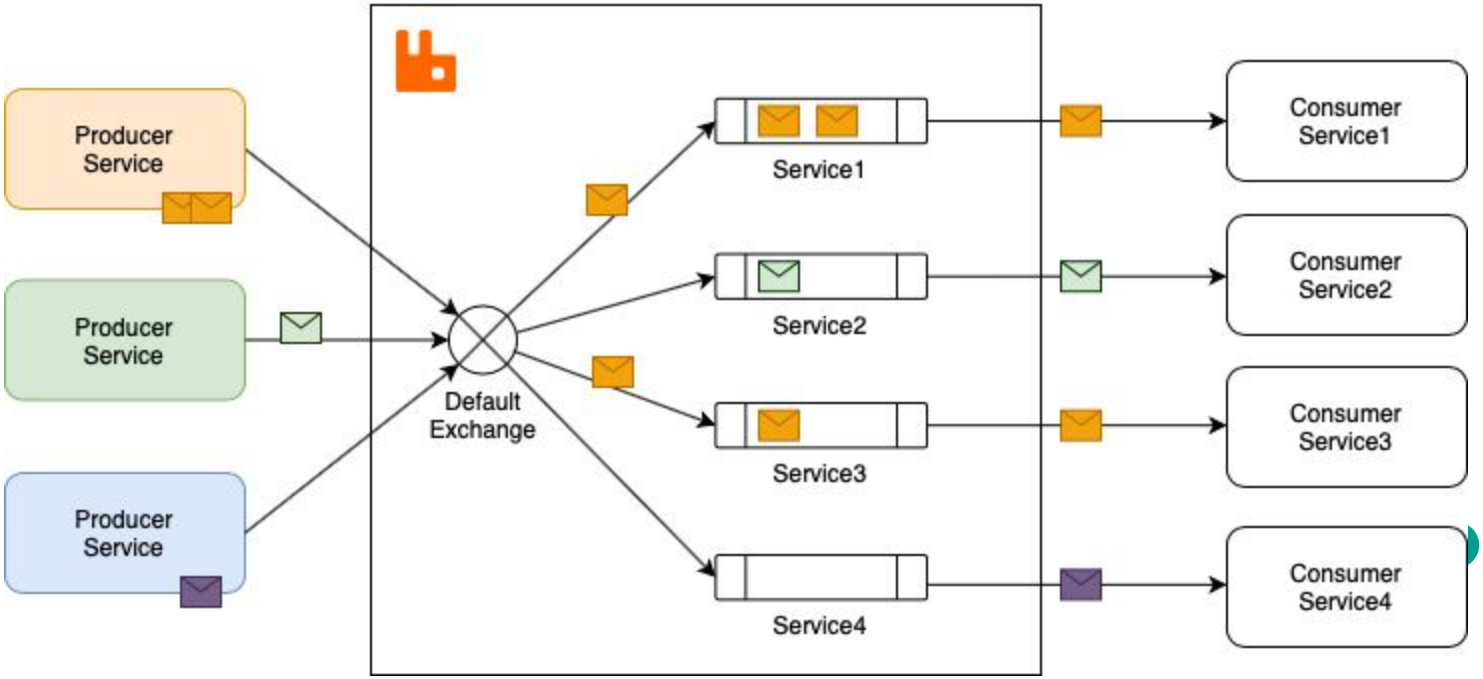
Event-Driven Architecture

Instead of connecting every node to every other node, connect all to one central bus.



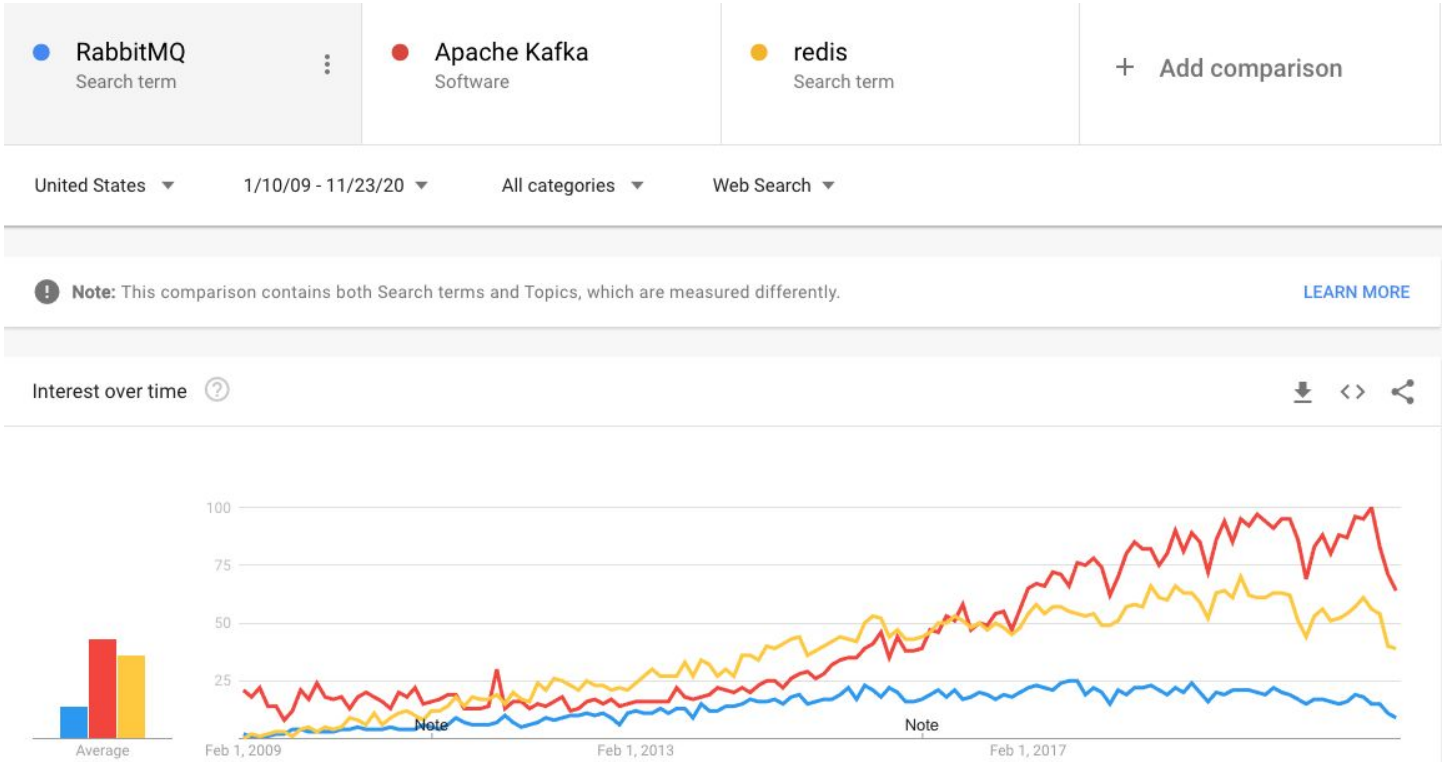
RabbitMQ

Open Source, written in Erlang



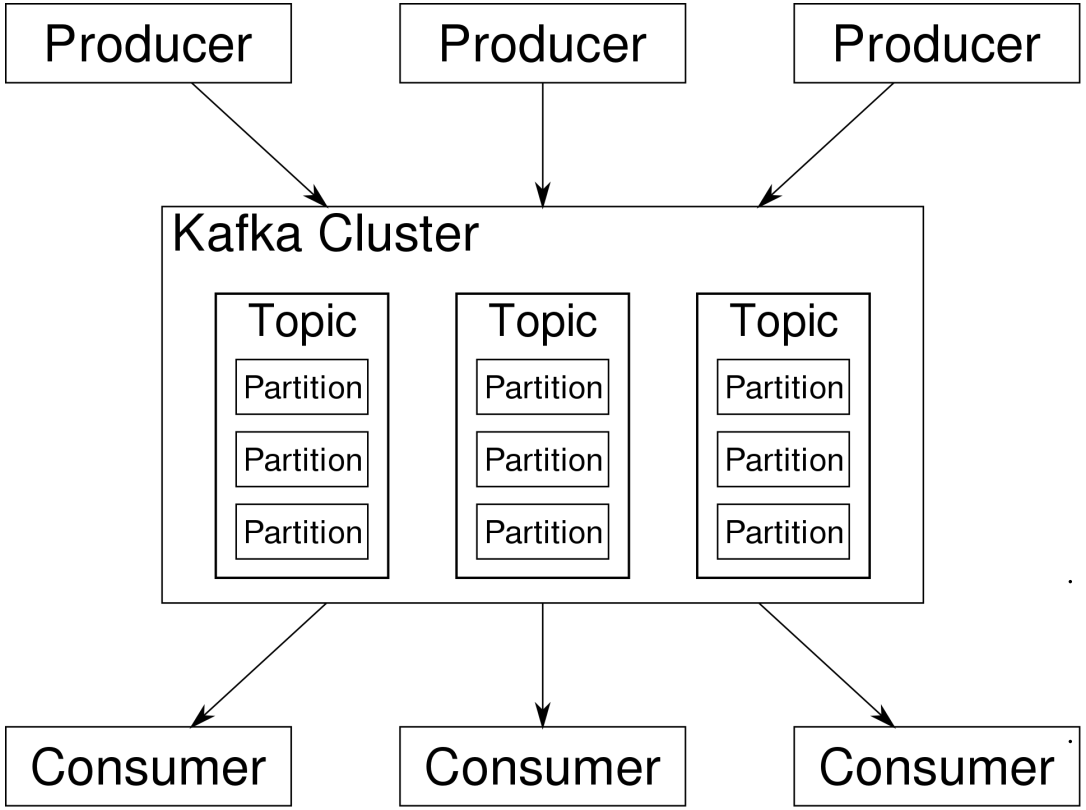
Technologies for Event Busses

RabbitMQ
Redis
ActiveMQ
Kafka



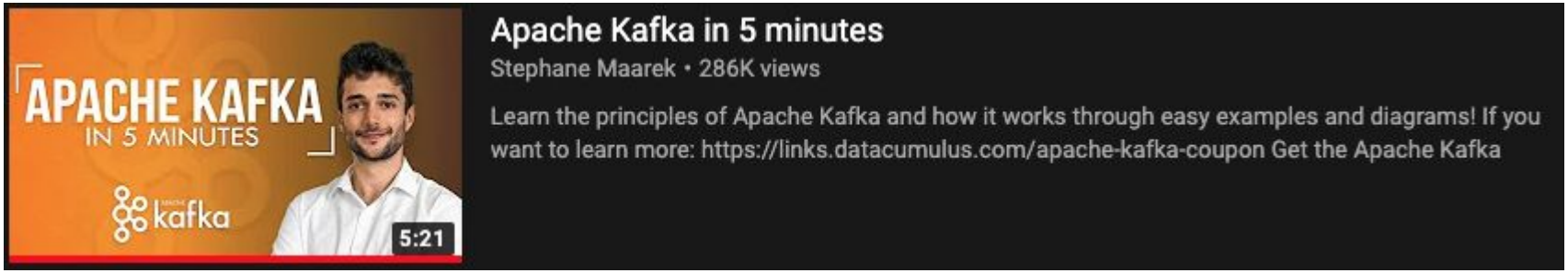
Kafka

Kafka was first developed by LinkedIn and released as open source in 2011. It is written in Scala an Java.

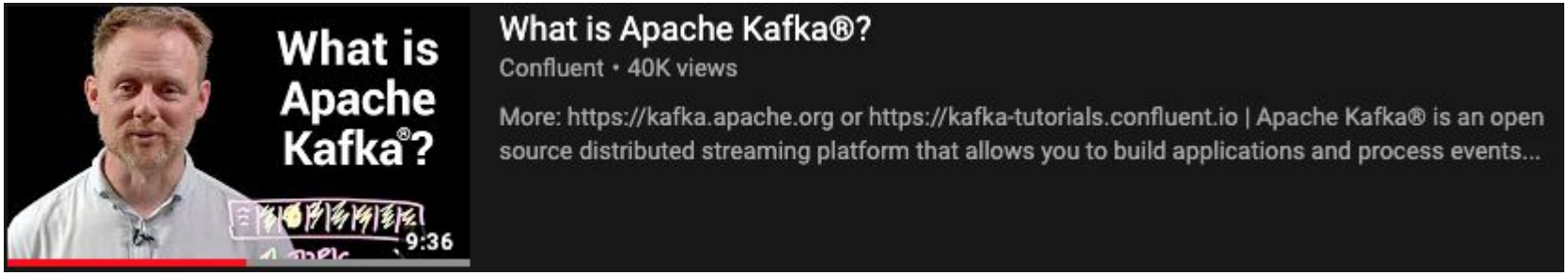


A short Intro on YouTube

Kafka in 5 Minutes



What is Kafka



Kafka API

Kafka has 4 main APIs. They are available in Java and Scala.

- Producer
  - Producers write Events to Kafka
- Consumer
  - Consumer subscribe to Events and read them
- Streams
- Connect

We will focus on Producer and Consumer.

Events

Events are the fundamental abstraction of Kafka. An Event has

- a key
- a value
- a timestamp
- metadata

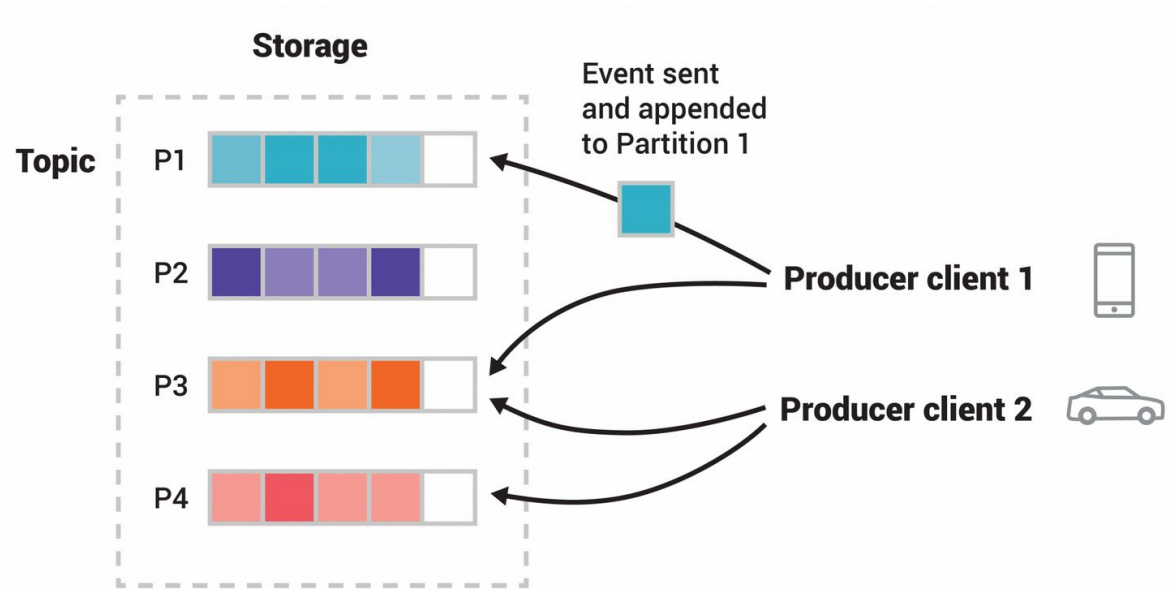
Events are not deleted after they are read, they are not consumed. They can be read by many subscribers.

Events are organized and stored in Topics

Topics and Brokers

Events in Topics are stored to hard drives. Topics can be configured to store events for a certain time (retention). Only after that time, Events are discarded.

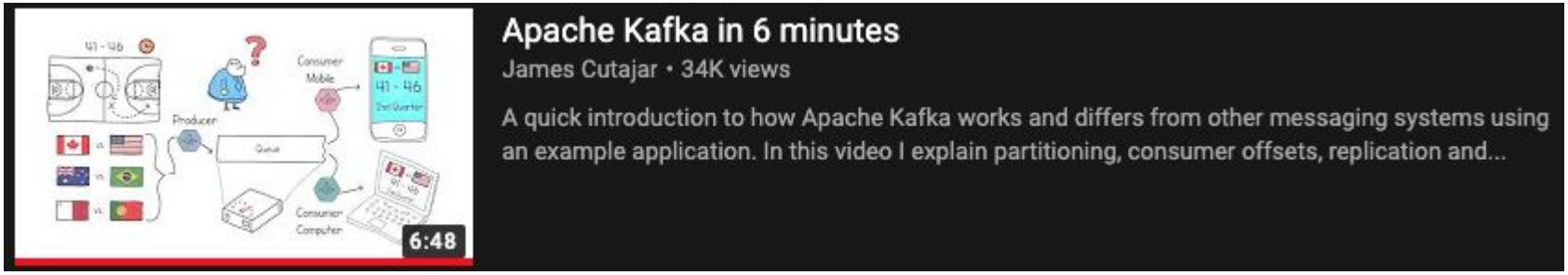
Topics are partitioned over several Brokers. Events with the same key are written to the same Partition. The order of the Events in one Partition is guaranteed.



Technical Aspects of Kafka

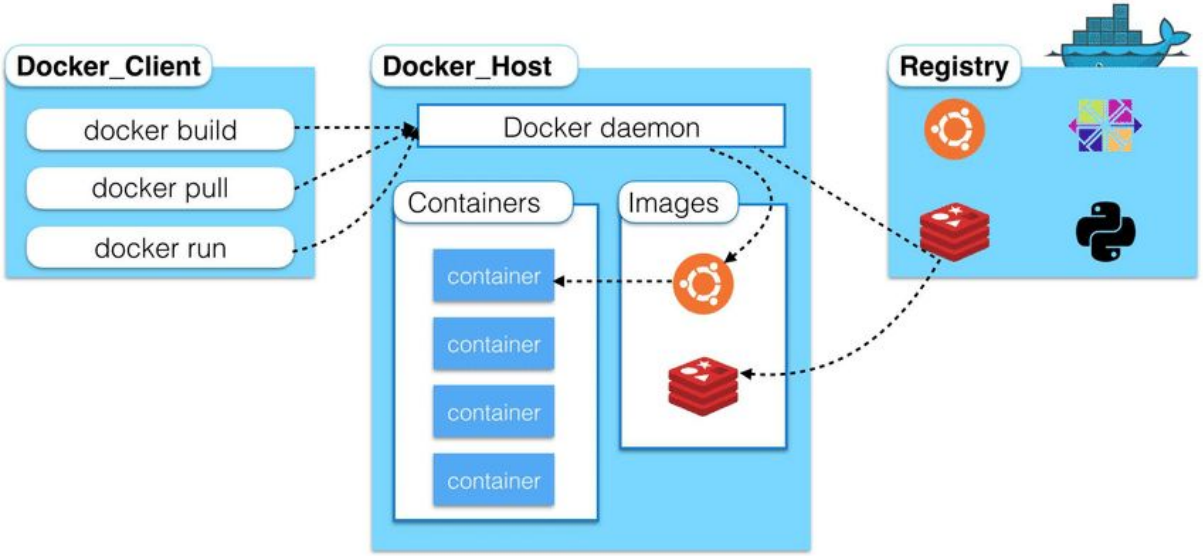
This video explains some more techical aspects of Kafka

https://www.youtube.com/watch?v=Ch5VhJzaoaI



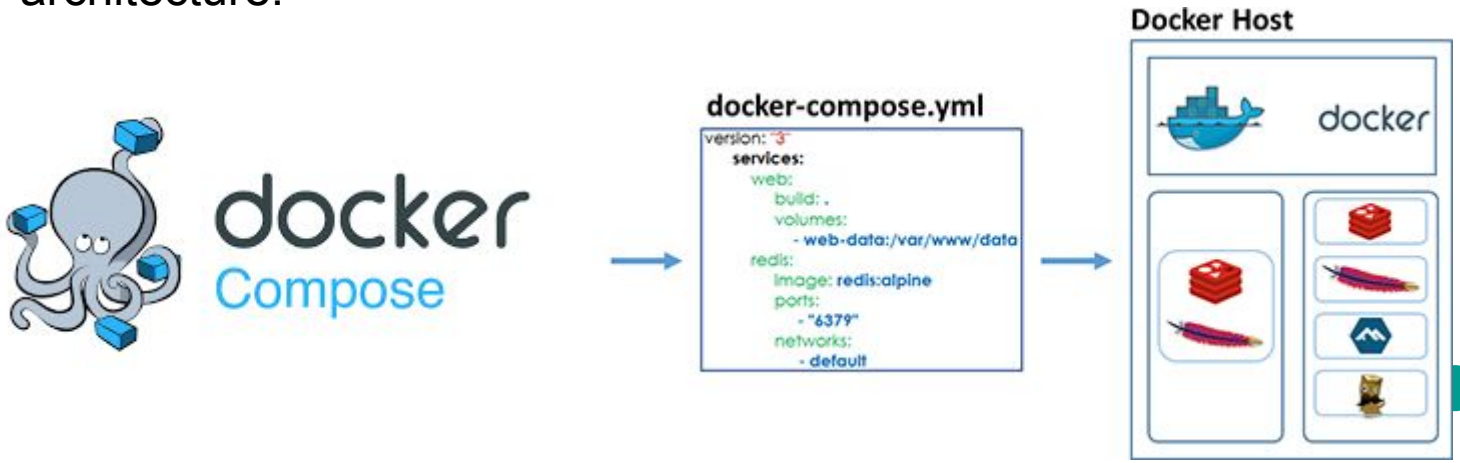
Docker

Docker is an OS-level virtualization software. Each virtual OS is called a Container. Containers are instantiated from Images. Images are shared on a Registy, for example DockerHub.



Docker Compose

Docker Compose is an orchestration tool that makes spinning up multi-container distributed applications with Docker an effortless task. It is easy to define a multi-container application and accompanying services in one file, then spin it up in a single command that will start and run your entire micro service architecture.

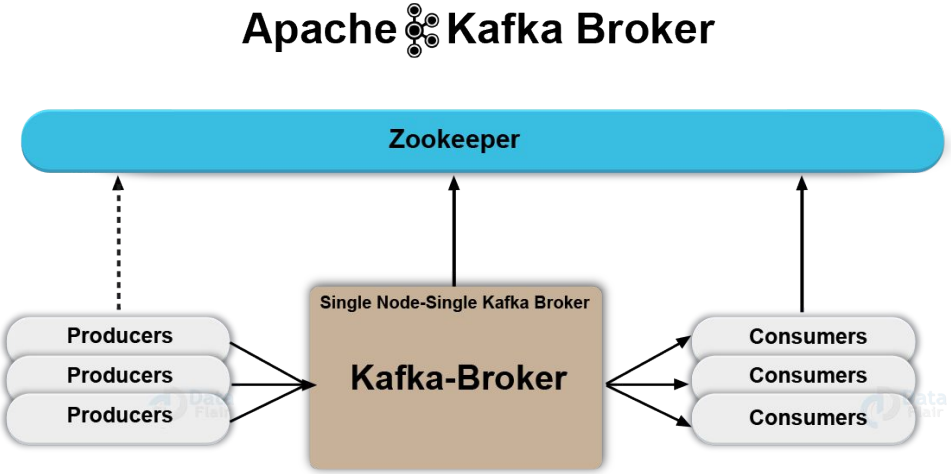


ZooKeeper

ZooKeeper is a centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services.

When working with Apache Kafka, ZooKeeper is primarily used to track the status of nodes in the Kafka cluster and maintain a list of Kafka topics and messages.

Specifically, ZooKeeper is used for controller election, cluster membership, topic configuration, access control lists, and quotas.



Kafka in Development

Kafka usually runs on a Cluster of Servers. Then it has fallback and fault tolerance. But for development we just want to access it.

Here is a Docker container that allows that:

https://github.com/simplesteph/kafka-stack-docker-compose

Run these commands

```
git clone https://github.com/simplesteph/kafka-stack-docker-compose.git
cd cd kafka-stack-docker-compose/
docker-compose -f zk-single-kafka-single.yml up
```

Then Kafka runs on local host on the default port

```
LISTENER_DOCKER_EXTERNAL://127.0.0.1:9092
```

Using Kafka from Scala

Using Kafka from Scala is a bit cumbersome, the documentation only provides the API in Java. The API can be used from Scala, but details can be hard to figure out.

Import the libraries from your build.sbt file

```
libraryDependencies += "org.apache.kafka" %% "kafka" % "2.6.0"
```

Producer

```
package de.htwg.scala.kafka

import java.util.Properties
import org.apache.kafka.clients.producer._

object Producer extends App {

 val  props = new Properties()
 props.put("bootstrap.servers", "localhost:9092")
 props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer")
 props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer")

 val producer = new KafkaProducer[String, String](props)

 val TOPIC="test"
 val KEY="message"

 for(i<- 1 to 50){
   val record = new ProducerRecord(TOPIC, KEY, s"hello $i")
   Thread.sleep(1000)
   println("Sending ..")
   producer.send(record)
 }

 val record = new ProducerRecord(TOPIC, KEY, "the end "+new java.util.Date)
 println("Sending final message..")
 producer.send(record)

 producer.close()
}
```

Consumer

```
package de.htwg.scala.kafka

import java.util
import java.util.Properties
import org.apache.kafka.clients.consumer.KafkaConsumer
import scala.collection.JavaConverters._

object Consumer extends App {

 val  props = new Properties()
 props.put("bootstrap.servers", "localhost:9092")
 props.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer")
 props.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer")
 props.put("group.id", "something")

 val consumer = new KafkaConsumer[String, String](props)
 val TOPIC="test"

 consumer.subscribe(util.Collections.singletonList(TOPIC))

 while(true){
   println("Polling..")
   val records=consumer.poll(100)
   for (record<-records.asScala){
     println(record)
   }
 }
}
```

Write a Producer and Consumer in Kafka

Use the provided stack to build a (set of) Producer and Consumer in Scala.

Kafka should be connected to your Micro Services using your Akka data stream.